PROGRAMMING FUNDAMENTALS

CS – 101

Instructor: Maria N. Samad

October 28th, 2022

LIST

- Sequence of values
- Can contain any type of data, called the *elements* or *items* of the list
- Ordered collection, supporting negative indexing
- Can be nested with multiple types of elements
- Preferable for shorter sequences of data, as the memory usage will increase with each new addition
- Can be homogenous or heterogenous
- Use square brackets to create lists in Python just like arrays
  - Numbers = [10, 20, 30, 40]
  - Animals = [“green frog”, “busy bee”, “slimy snake”]
  - Multiple = [“spam”, 2.0, 5, [10, 20]]

ACCESSING LIST

- Accessing list is similar to accessing characters of a string
  - Using the square brackets
  - Example:
    - cheeses = [“Cheddar”, “Feta”, “Gouda”]
    - numbers = [17, 123]
    - empty = [ ]
    - print(cheeses) → output = ?
    - Print(numbers) → output = ?
    - Print(empty) → output = ?
- Mapping

LISTS ARE MUTABLE

- Unlike Strings, individual elements can be modified in Lists
- For example:
  - Numbers[1] = 5
  - Chesses[2] = “Parmesan”

THE in OPERATOR

- Just like Strings, the ***in*** keyword can be used to search for an element in lists
- For example:
  - print('Cheddar' in cheeses) → Output?
  - print('Mozarella' in cheeses) → Output?

LIST TRAVERSAL

- Most common way to traverse a list is by using ***for*** loop
  - Traversing the elements
    - For example:

```
for cheese in cheeses:                    → Output?
        print (cheese)
```

  - Traversing the indices
    - For example:

```
for i in range(len(numbers)):             → Output?
        numbers[i] = numbers[i] * 2
```

- Traversing an empty list never executes the body of the loop
  - For example:

```
for x in [ ]:
    print ("It never comes in here!")
```